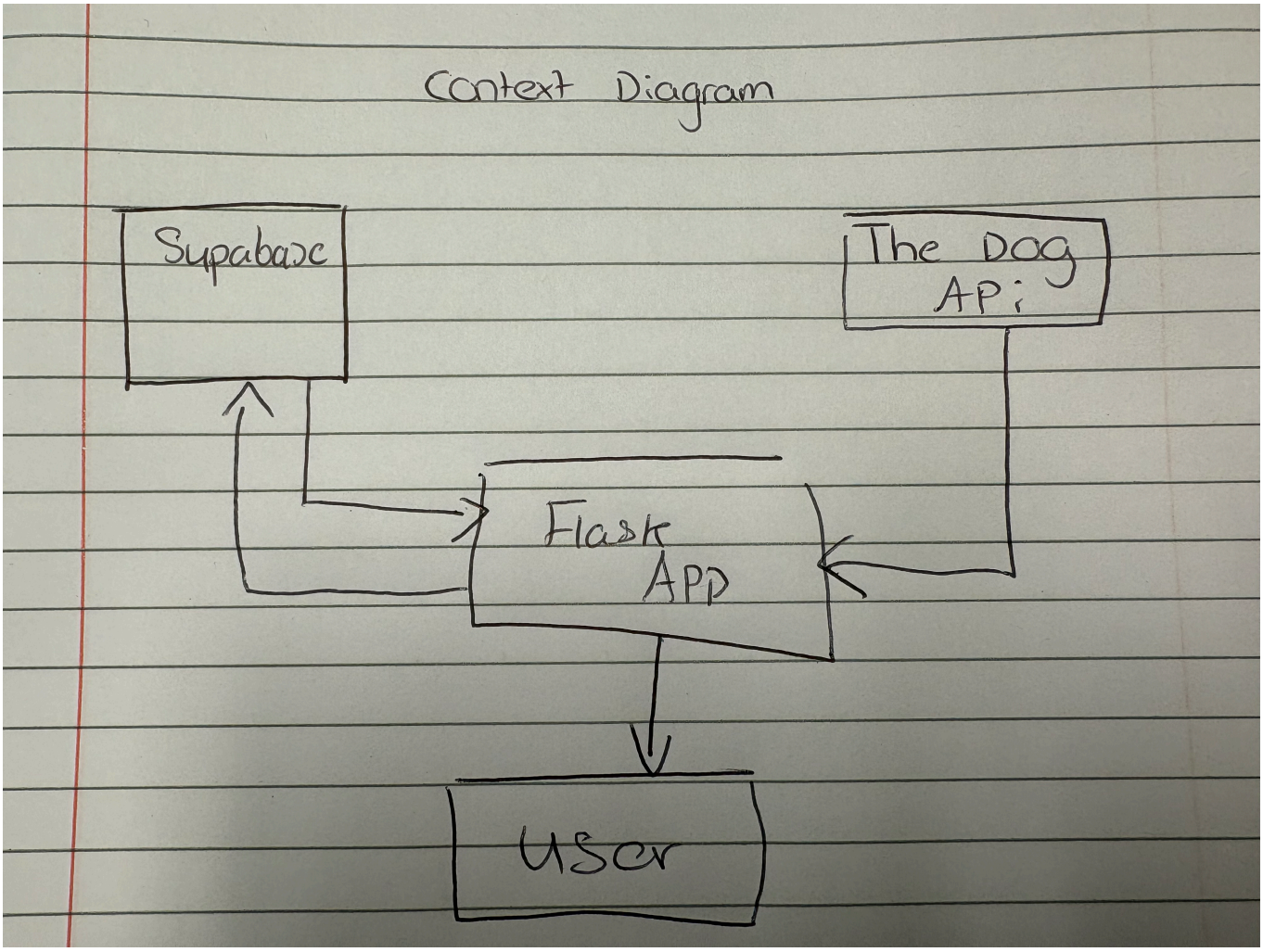
Architecture Note

DeployHub is a Flask web application that integrates two external services. a Supabase PostgreSQL database and The Dog API. This allow users to search for dog breeds, view detailed breed information, and retrieve a random breed. The application is deployed on Render and managed through GitHub with an automated CI/CD pipeline.

Context Diagram



The user interacts with the Flask app through a browser. The app first checks Supabase for the requested breed. If it is not found in supabase, it falls back to The Dog API. Results are stored back into Supabase for future requests. All user actions and system events are logged to a separate Supabase log table.

Integration Points

Supabase

- Used as the primary data store for dog breeds

- Stores all breed data retrieved from the API to reduce repeated external calls
- Also used as the logging backend — every user action and system event is inserted into a log table
- Connected via the official Supabase Python client using environment variables

The Dog API

- Used as the fallback when a breed is not found in Supabase
- Also used for the random breed feature which fetches all breeds and selects one at random
- Connected via HTTP requests using the requests library with an API key stored as an environment variable

Integration Flow

1. User searches "Labrador"
2. App checks Supabase dog_breeds table
    1. If found → return result to user
    2. If not found → call Dog API
        1. If API finds it → save to Supabase and return result
    3. If API finds nothing → show error message to user

Branching Model

We used a feature branch strategy throughout the project. All work was done on short-lived feature branches which were merged into main via pull requests. No one pushed directly to main after the initial project setup commit.

Over the course of the project approximately 20 branches were created, each representing a distinct piece of work such as: Feature/random-breed, feature/ci, feature/dockerfile.

Every feature branch was merged into main via a pull request. Branch protection rules were applied towards the end of the project requiring CI checks to pass before merging. In earlier PRs reviews were not always formally recorded on GitHub, however code was discussed and agreed upon by the team before merging. Going forward in a production environment formal approvals would be enforced from the start.

Pipeline Design

Our CI/CD pipeline was implemented using GitHub Actions and runs automatically on every pull request and every push to main. Pipeline steps in order:

1. Checkout code — pulls the latest code from the branch
2. Set up Python 3.11

3. Install dependencies — runs pip install -r requirements.txt
4. Lint with ruff — checks code for style and syntax issues
5. Run tests with pytest — For coverage reporting
6. Build Docker image — builds the container to confirm it compiles correctly
7. Push to GHCR — publishes the Docker image to GitHub Container Registry

Continuous Deployment: Render is connected directly to the GitHub repository and automatically redeploys the application whenever a new commit is merged to main. This means every approved and reviewed PR results in a live deployment with no manual steps required.

It is worth noting that the full CI pipeline including lint and tests was implemented towards the end of the project. Earlier PRs were merged with a more basic pipeline that only built the Docker image. In a production environment the full pipeline would be enforced from day one.

Endpoints

Endpoint	Purpose
/	Sets session, redirects to home
/home	Search for a dog breed / Select random breed
/breed/id	View detailed breed information
/random	Fetch and display a random breed
/health	JSON health check for monitoring
/status	Visual status page showing service connectivity
/logs	View application activity logs